

DBA-Fusion: Tightly Integrating Deep Dense Visual Bundle Adjustment With Multiple Sensors for Large-Scale Localization and Mapping

Yuxuan Zhou¹, Xingxing Li¹, Shengyu Li¹, Xuanbin Wang¹, Shaoquan Feng¹, and Yuxuan Tan¹

Abstract—Visual simultaneous localization and mapping (VSLAM) has broad applications, with state-of-the-art methods leveraging deep neural networks for better robustness and applicability. However, there is a lack of research in fusing these learning-based methods with multi-sensor information, which could be indispensable to push related applications to large-scale and complex scenarios. In this letter, we tightly integrate the trainable deep dense bundle adjustment (DBA) with multi-sensor information through a factor graph. In the framework, recurrent optical flow and DBA are performed among sequential images. The Hessian information derived from DBA is fed into a generic factor graph for multi-sensor fusion, which employs a sliding window and supports probabilistic marginalization. A pipeline for visual-inertial integration is firstly developed, which provides the minimum ability of metric-scale localization and mapping. Furthermore, other sensors (e.g., global navigation satellite system) are integrated for driftless and geo-referencing functionality. Extensive tests are conducted on both public datasets and self-collected datasets. The results validate the superior localization performance of our approach, which enables real-time dense mapping in large-scale environments.

Index Terms—Visual simultaneous localization and mapping, multi-sensor fusion, dense bundle adjustment.

I. INTRODUCTION

VISUAL simultaneous localization and mapping (VSLAM) is a pivotal technology in VR/AR and robotics applications [1]. Recently, deep learning has significantly propelled the advancement of VSLAM on better accuracy, robustness and dense spatial perception by utilizing higher-level information [2], [3]. Due to the data-driven nature, the generalization of end-to-end VSLAM systems on new data domains is challenging. To address this issue, DROID-SLAM [4] designs a recurrent structure integrating optical flow and dense bundle adjustment (DBA), which is end-to-end trainable and demonstrates excellent zero-shot performance.

Manuscript received 1 January 2024; accepted 7 May 2024. Date of publication 13 May 2024; date of current version 22 May 2024. This letter was recommended for publication by Associate Editor H. Saito and Editor P. Vasseur upon evaluation of the reviewers' comments. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3907100, and in part by the National Natural Science Foundation of China under Grant 42204017. (Corresponding author: Xingxing Li.)

The authors are with the School of Geodesy and Geomatics, Wuhan University, Wuhan 430070, China (e-mail: xxli@sgg.whu.edu.cn).

The code has been made open-source (<https://github.com/GREAT-WHU/DBA-Fusion>).

Digital Object Identifier 10.1109/LRA.2024.3400156

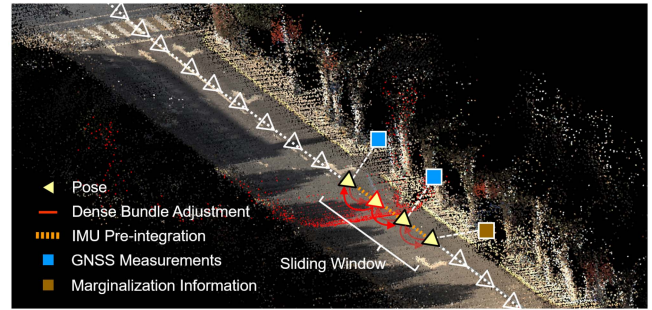


Fig. 1. Illustration of the DBA-Fusion system. The pointcloud map is generated online via monocular visual-inertial-GNSS integration. For clarity, we only show 5x downsampled camera poses.

It's important to note that, a visual-only system has its inherent limitations and is vulnerable to extreme illumination issues and dynamic scenarios. To address these challenges, a promising approach is to incorporate multiple sensors, such as inertial measurement units (IMU) [5], global navigation satellite system (GNSS) [6] and wheel speed sensors (WSS) [7] for optimal information fusion. Although the integration of VSLAM systems with multiple sensors has been widely studied, there is limited research when it comes to learning-based VSLAM frameworks. As model-driven methods are considered effective enough for the above mentioned sensors in most cases, the key challenge lies in how to probabilistically fuse multi-sensor state estimation with deep VSLAM methods, form a generic framework, and assess its practical significance in terms of state estimation and mapping.

In this work, we propose DBA-Fusion, which provides a generic factor graph framework to tightly integrate DROID-SLAM-like deep DBA with multi-sensor information, as illustrated in Fig. 1. The system is designed to enhance the practicality of deep VSLAM through multi-sensor fusion and make it suitable for real-time large-scale applications. The contributions of this work are summarized as follows:

- 1) We provide a pipeline for tight visual-inertial integration which introduces the recurrent deep DBA into a sliding-window factor graph optimization framework.
- 2) The proposed system supports flexible integration of multiple sensors (e.g. GNSS, WSS), thus can be applied to large-scale, georeferenced navigation and mapping applications.
- 3) The comprehensive performance of the system is evaluated on both public and self-made datasets covering indoor/outdoor, handheld/vehicular scenarios.

II. RELATED WORK

A. Deep VSLAM

Deep learning has been studied for integration into VSLAM in recent years with diverse motivations. Learning-based methods are used to replace the key modules in VSLAM, like feature tracking [8], depth inference [9] or to achieve object-level SLAM with semantic information [10], [11]. There are also many works that attempt to develop end-to-end visual SLAM systems [12], [13]. Recent works have been concentrating on the neural representation of 3-D spatial map, which has shown impressive reconstruction performance [14], [15].

Among the above methods, DROID-SLAM is a popular one which combines the advantages of traditional SLAM pipeline and end-to-end learning, and is well-known for its good generality and dense mapping capability. The core design of DROID-SLAM is a recurrent optical flow module based on gate recurrent units (GRU) and a differentiable DBA layer, which makes it decoupled with depth inference and can be flexibly applied to zero-shot datasets.

B. VSLAM With Multi-Sensor Fusion

Visual-inertial integration is a commonly used scheme to overcome the limitation of visual-only SLAM, which provides scale awareness and estimation continuity with an almost minimum setup. The classic implementations could be divided into filter-based [16] and optimization-based [17]. Advanced features have been developed over the past decades, including observability constraint [18], direct photometric optimization [19], map management [20] and delayed marginalization [21]. In [22], DROID-SLAM is fused with IMU in a factor graph framework. However, marginalization and monocular setup are not considered. Recently, end-to-end visual-inertial SLAM structures have also been proposed [23].

To expand the applicability of the V-I system, many other works tend to fully utilize the information of available sensors, of which the most representative ones are wheel encoder for ground vehicles and GNSS for outdoor applications. The wheel encoder has been proved to improve the system stability and scale observability [7][24]. As to GNSS integration, both loosely coupled [25] and tightly coupled [6], [26] methods have been proposed, which generally elevate the system to a geo-registered one and provide global driftless capability. Nevertheless, there is a noticeable absence of an implementation that incorporates advanced learning-based VSLAM into the multi-sensor fusion framework.

III. SYSTEM IMPLEMENTATION

The overall design of the framework aims to flexibly fuse a trainable VSLAM system with multi-sensor information.

For the visual frontend, a recurrent optical flow module is utilized to compute dense pixel association for image pairs in a co-visibility graph. The re-projections are used to formulate a DBA problem. The DBA information is then tightly fused with multi-sensor through a factor graph, where a sliding window mechanism with probabilistic marginalization is employed.

As to multi-sensor fusion, IMU is firstly considered in the factor graph to enable a minimum visual-inertial odometry (VIO).

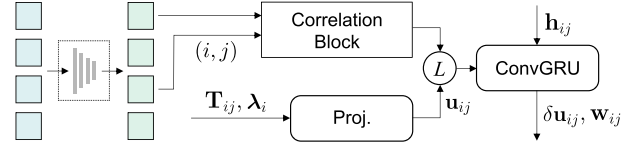


Fig. 2. Illustration of the recurrent optical flow. The images are firstly encoded to feature maps. One co-visible image pair (i, j) is fed into the recurrent optical flow module, as described in Section III-A.

Besides, to improve the system applicability in large-scale scenarios, common sensors like GNSS and WSS are integrated into the factor graph when available.

A. Recurrent Optical Flow

The recurrent optical flow applied in this work adheres to the implementation of DROID-SLAM, which is derived from the well-developed RAFT [27] and tightly leverages the camera poses and depths for iterative refinement.

The structure of the recurrent optical flow is depicted in Fig. 2. To be specific, for a co-visible image pair (source frame i projected to target frame j), a correlation volume is formed based on their encoded feature maps. During one update operation of the optical flow, firstly, the (inverse) depth map λ_i of the source frame is projected via the relative pose $\mathbf{T}_{ij}$ to estimate the initial optical flow field

$$\mathbf{u}_{ij} = \Pi_c(\mathbf{T}_{ij} \circ \Pi_c^{-1}(\mathbf{u}_i, \lambda_i)), \quad \mathbf{T}_{ij} = \mathbf{T}_j \circ \mathbf{T}_i^{-1} \quad (1)$$

where $\mathbf{u}_i$ is the pixel coordinates of the grid points, $\mathbf{T}_i$ and $\mathbf{T}_j$ are world-to-camera transformations, Π_c is the camera projection model, whose inverse operation uplifts the pixel coordinates and inverse depths to 3-D points. Notice that (1) is also used to estimate inter-frame disparity to check the co-visibility among frames in further steps.

Based on the flow estimation, a look-up operation L to the correlation volume is performed. The results are then injected into the convolutional GRU with hidden state $\mathbf{h}_{ij}$ to obtain the corrected flow $\delta \mathbf{u}_{ij}$ in the residual form. Besides, the pixel-wise weight $\mathbf{w}_{ij}$ of the optical flow, indicating geometric and appearance uncertainty, is generated to boost the downstream BA. For real-time performance, we only consider pixels in each image with a step size of 8. In this case, the size n of λ_i is $(H/8) \times (W/8)$, where (H, W) is the size of the image.

The above optical flow is computed on every active image pair (termed as an edge) in the co-visibility graph, thus to construct a DBA problem. The optimization results of DBA, i.e. the dense depths and camera poses, are recurrently fed into the optical flow module to refine everything iteratively. Such operations would be performed continuously to adjust the optical flow until achieving a converged state, which is different from the usual forward-tracking visual frontend. The recurrent optical flow design has been verified to show good generalization on zero-shot datasets. In this work, we just use the network weights pretrained on the synthetic TartanAir [28] dataset for all the tests later.

B. Integrating DBA Into a Generic Factor Graph

The basic element of DBA is one uni-directional edge in the co-visibility graph, corresponding to the re-projection errors from one source frame to one target frame, whose linearized

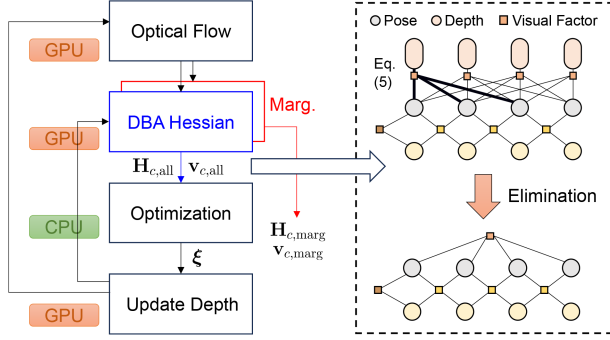


Fig. 3. Workflow of DBA in the optimization framework, which employs a hierarchical iteration structure. The right panel shows the elimination of the depth state, which generates the Hessian $\mathbf{H}_{c,all}$ and $\mathbf{v}_{c,all}$ as the visual constraint. For marginalization, only related edges are processed to generate $\mathbf{H}_{c,marg}$, $\mathbf{v}_{c,marg}$ that contains the marginalized visual information.

form is

$$\underbrace{(\delta \mathbf{u}_{ij})_{2n \times 1}}_{\mathbf{r}} = \underbrace{[\mathbf{J}_i \quad \mathbf{J}_j \quad \mathbf{J}_{\lambda_i}]}_{\mathbf{J}} \underbrace{\begin{bmatrix} \xi_i^\top & \xi_j^\top & (\delta \lambda_i)^\top \end{bmatrix}}_{\mathbf{x}} \quad (2)$$

where $\delta \mathbf{u}_{ij}$ is the optical flow residual outputted by the optical flow module, ξ_i, ξ_j are Lie algebras of the camera poses, $\delta \lambda_i$ is the error state of the source frame's inverse depth map, $\mathbf{J}_i, \mathbf{J}_j, \mathbf{J}_{\lambda_i}$ are Jacobians derived from (1).

Taking into account the weight w_{ij} of the optical flow, the Hessian form of (2) is computed following

$$\mathbf{J}^\top \mathbf{W} \mathbf{r} = \mathbf{J}^\top \mathbf{W} \mathbf{J} \mathbf{x}, \quad \mathbf{W} = \text{diag}(w_{ij}) \quad (3)$$

and leads to the following equation

$$\begin{bmatrix} \mathbf{v}_{ii} \\ \mathbf{v}_{ij} \\ \mathbf{z}_{ii} \end{bmatrix} = \begin{bmatrix} \mathbf{B}_{ii} & \mathbf{B}_{ij} & \mathbf{E}_{ii} \\ \mathbf{B}_{ij}^\top & \mathbf{B}_{jj} & \mathbf{E}_{ij} \\ \mathbf{E}_{ii}^\top & \mathbf{E}_{ij}^\top & \mathbf{C}_{ii} \end{bmatrix} \begin{bmatrix} \xi_i \\ \xi_j \\ \delta \lambda_i \end{bmatrix} \quad (4)$$

where $\mathbf{C}_{ii}$ is a $n \times n$ diagonal matrix.

Considering a bundle of edges anchored on frame i and projected to co-visible frames (numbered 1, 2, ..., N as example), the combined Hessian is constructed by positionally stacking/summing the blocks in (4) as

$$\begin{bmatrix} \Sigma \mathbf{v}_{ii} \\ \mathbf{v}_{i1} \\ \vdots \\ \mathbf{v}_{iN} \\ \Sigma \mathbf{z}_{ii} \end{bmatrix} = \begin{bmatrix} \Sigma \mathbf{B}_{ii} & \mathbf{B}_{i1} & \cdots & \mathbf{B}_{iN} & \Sigma \mathbf{E}_{ii} \\ \mathbf{B}_{i1}^\top & \mathbf{B}_{11} & & & \mathbf{E}_{i1} \\ \vdots & & \ddots & & \vdots \\ \mathbf{B}_{iN}^\top & & & \mathbf{B}_{NN} & \mathbf{E}_{iN} \\ \Sigma \mathbf{E}_{ii}^\top & \mathbf{E}_{i1}^\top & \cdots & \mathbf{E}_{iN}^\top & \Sigma \mathbf{C}_{ii} \end{bmatrix} \begin{bmatrix} \xi_i \\ \xi_1 \\ \vdots \\ \xi_N \\ \delta \lambda_i \end{bmatrix} \quad (5)$$

which corresponds to one visual factor depicted in Fig. 3. Note that no other edges in the graph are related to $\delta \lambda_i$, thus Schur complement [29] can be performed on (5) to eliminate the depth state, leading to

$$\underbrace{(\mathbf{B}_i - \mathbf{E}_i \mathbf{C}_i^{-1} \mathbf{E}_i^\top)}_{\mathbf{H}_{c,i}} \xi_{i,1,\dots,N} = \underbrace{\mathbf{v}_i - \mathbf{E}_i \mathbf{C}_i^{-1} \mathbf{z}_i}_{\mathbf{v}_{c,i}} \quad (6)$$

where $\mathbf{B}_i, \mathbf{E}_i, \mathbf{C}_i, \mathbf{v}_i, \mathbf{z}_i$ are blocks in (5) and $\mathbf{C}_i$ is a diagonal matrix. This actually constructs a inter-frame pose constraint (represented by $\mathbf{H}_{c,i}, \mathbf{v}_{c,i}$) which contains the linearized BA information. The calculation of (5) and (6) can be efficiently

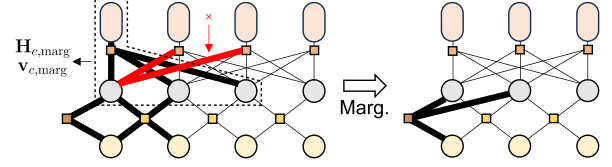


Fig. 4. Marginalization of the factor graph. Visual information as well as other information related to the oldest states (black edges) is marginalized, which uses Schur complement to eliminate the oldest states and form a new constraint [17], [29]. The re-projections from newer frames to the old frame (red edges) are neglected to keep sparsity.

performed in parallel on GPU. As long as the camera poses are updated, the inverse depth state can be updated according to the bottom row of (5)

$$\delta \lambda_i = \mathbf{C}_i^{-1} (\mathbf{z}_i - \mathbf{E}_i^\top \xi_{i,1,\dots,N}) \quad (7)$$

In this work, in order to tightly integrate the DBA with a generic pose-centered factor graph, all the constraints (6) derived from the co-visibility graph are stacked to form a full Hessian $\mathbf{H}_{c,all}$ and corresponding $\mathbf{v}_{c,all}$, which is taken as a constraint among all camera poses in the factor graph. The factor graph optimization is efficiently performed on CPU, considering that the dense depth states have been eliminated. This is actually equal to solving the full factor graph optimization in a two-step way, similar to the implementation in [21]. The pixel association (produced by the optical flow) and the DBA formulation need to be iteratively updated for edges in the graph, leading to the hierarchical iteration structure shown in Fig. 3. Generally, the deep DBA provides rich geometric information (with learned uncertainty) to the factor graph, while the optimization results (poses and depths) are fed back to refine/re-weight the optical flow recurrently. The above pipeline also supports stereo and RGB-D setups, either by adding left-to-right edges or adding depth priors in (5).

Every time an old frame is slid out of the window, the camera pose and the contained depth state need to be marginalized. To keep the sparsity of the factor graph, only the re-projection residuals built upon the marginalized depth state are considered, which forms the partial visual information $\mathbf{H}_{c,marg}, \mathbf{v}_{c,marg}$ as shown in Fig. 4. The visual information and other information related to the oldest frame, together with the existing marginalization factor, are used to constitute the new marginalization factor through Schur complement.

C. Multi-Sensor Factor Graph

Applying the above design, we can build up a generic pose-centered factor graph to solve the pose estimation, with the advantages of efficient optimization and loseless DBA information integration. The factor graph optimization is implemented using GTSAM [30].

Taking the commonly used sensors in vehicle application into account, the overall factor graph is depicted in Fig. 5, with the states expressed as

$$\mathbf{x}_k = [\mathbf{T}_{b_k}^w \quad \mathbf{v}_{b_k}^w \quad \mathbf{b}_k], \quad k = 1, 2, 3, \dots \quad (8)$$

$$\mathbf{T}_{b_k}^w = \begin{bmatrix} \mathbf{R}_{b_k}^w & \mathbf{t}_{b_k}^w \\ 0 & 1 \end{bmatrix} \in SE(3), \quad \mathbf{b}_k = [\mathbf{b}_{a,k} \quad \mathbf{b}_{g,k}] \quad (9)$$

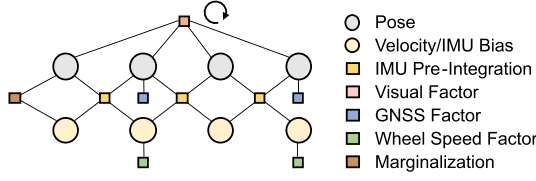


Fig. 5. The generic pose-centered factor graph that is used for multi-sensor fusion. Note that the visual factor is modeled using a Hessian factor where the point depths are temporarily eliminated. The visual factor is iteratively updated with the recurrent optical flow and DBA Hessian formulation.

where $\mathbf{T}_{b_k}^w$ is the IMU pose in the world frame, $\mathbf{v}_{b_k}^w$ is the velocity, $\mathbf{b}_{a,k}$ and $\mathbf{b}_{g,k}$ are accelerometer/gyroscope biases.

The sensor factors are presented as follows.

1) Visual constraint factor (quadratic cost function [30]):

$$\mathbf{E}_c(\mathbf{x}_c) = \frac{1}{2} \mathbf{x}_c^\top \mathbf{H}_{c,\text{all}} \mathbf{x}_c - \mathbf{x}_c^\top \mathbf{v}_{c,\text{all}} \quad (10)$$

where $\mathbf{H}_{c,\text{all}}$ and $\mathbf{v}_{c,\text{all}}$ are the information matrix and vector of DBA with all the depth parameters eliminated, $\mathbf{x}_c$ is the Lie algebras of the camera poses $\mathbf{T}_w^c$

$$\mathbf{x}_c = [\xi_w^{c_0^\top} \quad \xi_w^{c_1^\top} \quad \dots \quad \xi_w^{c_k^\top}]^\top \quad (11)$$

In order to impose the visual constraint on the IMU-centric factor graph, the following linear transformation is employed

$$\xi_w^c = -\text{adj}(\mathbf{T}_b^c) \xi_b^w \quad (12)$$

where ξ_w^c is derived from left-perturbation and ξ_b^w is derived from right-perturbation, $\mathbf{T}_b^c$ is the IMU-camera extrinsics, $\text{adj}(\cdot)$ is the adjoint operation.

2) IMU preintegration factor:

$$\mathbf{r}_b(\mathbf{x}_k, \mathbf{x}_{k+1}) = \begin{bmatrix} \mathbf{R}_{b_k}^{w^\top} \left(\mathbf{p}_{b_{k+1}}^w - \mathbf{p}_{b_k}^w + \frac{1}{2} \mathbf{g}^w \Delta t_k^2 - \mathbf{v}_{b_k}^w \Delta t_k - \Delta \tilde{\mathbf{p}}_{b_{k+1}}^{b_k} \right) \\ \mathbf{R}_{b_k}^{w^\top} \left(\mathbf{v}_{b_{k+1}}^w + \mathbf{g}^w \Delta t_k - \mathbf{v}_{b_k}^w \right) - \Delta \tilde{\mathbf{v}}_{b_{k+1}}^{b_k} \\ \text{Log} \left(\left(\mathbf{R}_{b_k}^w \right)^{-1} \mathbf{R}_{b_{k+1}}^w \left(\Delta \tilde{\mathbf{R}}_{b_{k+1}}^{b_k} \right)^{-1} \right) \\ \mathbf{b}_{a,k+1} - \mathbf{b}_{a,k} \\ \mathbf{b}_{g,k+1} - \mathbf{b}_{g,k} \end{bmatrix} \quad (13)$$

where $\Delta \tilde{\mathbf{p}}_{b_{k+1}}^{b_k}$, $\Delta \tilde{\mathbf{v}}_{b_{k+1}}^{b_k}$, $\Delta \tilde{\mathbf{R}}_{b_{k+1}}^{b_k}$ are the IMU preintegration terms [31], $\mathbf{g}^w$ is the gravity vector, Δt_k is the time interval. The integration of IMU and DBA enables the functionality of a VIO, while certain steps are needed to bootstrap the system.

3) GNSS position factor:

$$\mathbf{r}_g(\mathbf{x}_k) = \mathbf{T}_w^n \circ \mathbf{T}_{b_k}^w \circ \mathbf{t}_g^b - \tilde{\mathbf{p}}_g^n \quad (14)$$

where $\tilde{\mathbf{p}}_g^n$ is the measured position of the GNSS phase center in the navigation frame, $\mathbf{t}_g^b$ is the IMU-GNSS lever-arm, $\mathbf{T}_w^n$ is a fixed world-to-navigation transformation obtained through the initial alignment, as described in Section III-E.

4) Wheel speed factor:

$$\mathbf{r}_s(\mathbf{x}_k) = (\mathbf{R}_{w_k}^{b_k} \mathbf{v}_{b_k}^w + \boldsymbol{\omega}_{wb}^b \times \mathbf{t}_s^b)_y - \tilde{v}_s \quad (15)$$

where $\boldsymbol{\omega}_{wb}^b$ is the angular velocity of the IMU, $\mathbf{t}_s^b$ is the IMU-wheel lever-arm, $\tilde{v}_s$ is the measured wheel speed. Here we

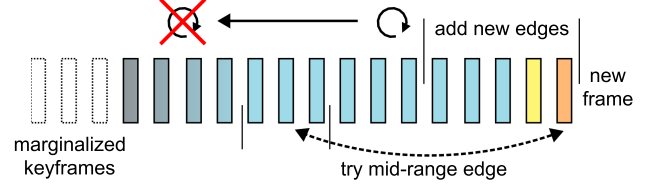


Fig. 6. Illustration of keyframe/edge management of DBA. New co-visible edges are appended within a small window and sparse mid-range edges are heuristically added. For mature edges that undergo enough iterations, the recurrent optical flow is no more performed. In our implementation, the maximum number of active edges (for optical flow update) is 48, the window size for DBA is 15, the range for co-visible edge appending is 5, the mid-range edges are tried between the newest frame and the frames indexed at (-8,-9,-10) relative to it.

apply a simplified wheel speed model, while it is convenient to introduce other parameters like the scale factor.

Besides, the marginalization factor is always taken into account, which contains the historical information. The generation of the marginalization factor is illustrated in Fig. 4. Note that the GNSS position factor and wheel speed factor related to the oldest states are also considered in the marginalization but are not depicted.

The generic design of the navigation-centered factor graph makes it flexible for different sensor schemes. Apart from the above factors, other common sensors, like GNSS doppler, magnetometer and specific motion models, could be easily added to the factor graph. In later discussions, we would pay most of the attention to the performance of a minimum monocular VIO system and a GNSS-integrated multi-sensor system for geo-related applications.

D. Other Details

1) Keyframe and edge management

In VO/VSLAM implementations, the selection and management of keyframes is important to maximize the estimation accuracy and reduce computation costs. Besides, the dense-indirect BA structure of the system necessitates extra considerations for edge addition/deletion in the graph, which affects the strength of visual association.

In our system, everytime a new frame comes (termed as one new epoch), the co-visibility among frames is checked to append the new edges, and the update operation (optical flow + optimization) of the graph is performed 2 times. If the coming image is with enough disparity, the second newest frame is retained as the new keyframe. In this case, another iteration of update is performed, and the oldest keyframe is marginalized. Otherwise, the second newest frame is abandoned.

For edge management, we only append new co-visible edges within a small window of relatively new keyframes, as shown in Fig. 6. In DROID-SLAM, a large window (25 by default) is used to check the co-visibility, thus to achieve better local consistency by introducing “local loop closure”. However, we find this uneconomical in our case and leads to high computational burden when the edges are dense. It is found that, with the aid of IMU, a small window for edge appending is enough to maintain high-accuracy and locally consistent pose estimation. To maximize the accuracy, we still heuristically add mid-range edges with high co-visibility in the graph and control the number of mid-range edges no more than 1 for a keyframe, which effectively reduces the peak computation load. The maximum

number of the active edges (which need optical flow update) in the graph is 48, which are prioritized for relatively new edges. For mature edges in the graph that undergo enough iterations, the optical flow is fixed while the re-projections are still considered in DBA until marginalized.

2) Visual-inertial initialization

As the IMU provides the awareness of scale and gravity direction, it is necessary to initialize the states in the metric-scale local frame for good convergence. In this work, we simply use the initialization method proposed in VINS-Mono [5], where the structure-from-motion is replaced by visual-only DBA. The DBA is iteratively performed based on the V-I initialization results, and the V-I translational extrinsic is taken into account after the first iteration. It is found that, with the accurate pose estimation given by DBA, the V-I initialization can be finished with high precision under common maneuver.

3) GNSS integration

As a global positioning technique, the position information provided by GNSS is expressed in earth-centered-earth-fixed (ECEF) frame. In this work, we simply align the trajectory produced by VIO to the GNSS trajectory to estimate the 4-DOF world-to-navigation transformation $\mathbf{T}_w^n$ then just fix it

$$\hat{\mathbf{T}}_w^n = \arg \min_{\mathbf{T}_w^n} \sum \|\mathbf{T}_w^n \circ \mathbf{T}_b^w \circ \mathbf{t}_g^b - \tilde{\mathbf{p}}_g^n\|_2 \quad (16)$$

where $\mathbf{T}_w^n$ comprises of a translation $\mathbf{t}_w^n$ and a heading θ_w^n . The navigation frame is defined as the east-north-up frame anchored at a specific location, and the navigation-to-ECEF transformation $\mathbf{T}_n^e$ is known based on WGS84. The alignment is performed as long as the travelled distance in the window exceeds the threshold (e.g., 10 m).

This strategy is simple and effective to integrate the GNSS information into the V-I system without introducing a local-to-global transformation state. The uncertainty of the initial alignment is transferred to the pose uncertainty, which would be continuously refined considering the full observability of GNSS measurements.

IV. EXPERIMENTS

In this part, we firstly test the monocular VIO pipeline of the system, termed as DBA-VIO, on the public TUM-VI and KITTI-360 datasets. After that, we test the multi-sensor integration performance based on the self-made vehicular dataset collected in urban environments.

For DROID-SLAM, DBA-VIO and DBA-Fusion, we use images at 5 Hz and scaled to 384×512 (or equivalent level) to guarantee real-time performance, which is verified sufficient for good results even in drastic motions. For navigation performance, we use real-time outputs of the system for evaluation, and the IMU-predicted poses are used for skipped frames if available. For dense mapping, the depth maps of marginalized keyframes are accumulated based on the estimated poses, and a multi-frame depth checking module [4] is used to filter out dynamic objects and outliers.

A. TUM-VI Dataset

The TUM-VI benchmark [32] is a visual-inertial dataset based on a handheld sensor platform, covering both indoor and outdoor scenarios. The platform comprises a stereo camera and a low-cost BMI160 IMU. The “magistrale” and “outdoors” sequences

TABLE I
ABSOLUTE TRANSLATION ERRORS (M) OF DIFFERENT VIO SCHEMES ON TUM-VI DATASET

Seq.	VINS stereo	VINS ¹ mono	ORB ² mono	DMVIO ¹ mono	DROID* mono	DBA mono	Leng. (m)
magi1	2.82	2.19	5.65	2.35	5.28	1.47	918
magi2	3.86	3.11	0.72	2.24	5.53	1.81	561
magi3	3.44	0.40	4.89	1.69	38.82	1.02	566
magi4	4.76	5.12	3.40	1.02	8.36	0.60	688
magi5	1.79	0.85	2.94	0.73	6.72	1.56	458
magi6	2.87	2.29	1.30	1.19	1.77	1.05	771
outd1	49.76	74.96	70.79	123.2	220.0	13.85	2656
outd2	17.10	133.5	14.98	12.76	60.55	12.79	1601
outd3	8.31	36.99	39.63	8.92	44.06	8.68	1531
outd4	13.74	16.46	25.26	15.25	26.12	4.45	928
outd5	21.19	130.6	19.38	7.16	14.00	3.73	1168
outd6	45.15	133.6	16.84	34.86	71.64	19.55	2045
outd7	19.38	21.90	19.68	5.00	17.42	4.18	1748
outd8	16.26	83.36	27.88	2.11	21.02	7.25	986
ave(%)	1.06	3.20	1.30	0.85	2.64	0.42	norm.

* Visual-only. ¹ Statistics borrowed from [21].

² Statistics partly borrowed from [20]. Sequence magi[3,6], outd[1-4,6,8] which were reported loop closure are re-run with loop detection disabled. Best results in bold.

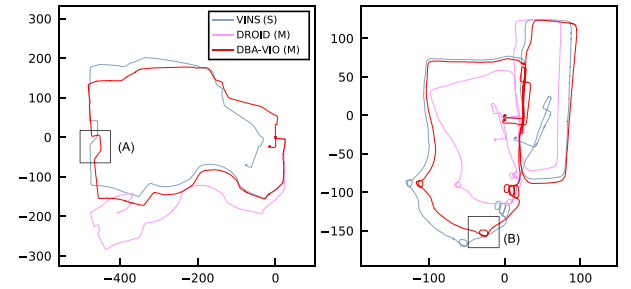


Fig. 7. Trajectories of typical odometry schemes on “outdoors1” and “outdoors7”. The trajectories are aligned with the ground-truth poses at the beginning of the sequence. The trajectory of DROID-SLAM (M) is scaled.

are selected for evaluation due to the challenging environment and relatively large scale. The ground truth poses from motion capture system are available for the start and end segments of the sequences. On this basis, the absolute translation error (ATE) is calculated via the evo toolkit [34], which indicates the overall odometry drift in this case.

Apart from DBA-VIO, different monocular VIO algorithms are taken into account for comparison, including VINS-Fusion (mono), ORB-SLAM3 (mono) and DM-VIO (mono). Besides, we consider VINS-Fusion (stereo) and DROID-SLAM (mono) for extra reference. For ORB-SLAM3 and DROID-SLAM, we disable loop detection/global optimization to fairly evaluate the odometry accuracy. For DROID-SLAM (mono) which is scale-less, we use the ground-truth poses available at the beginning to obtain a scale estimation, then use the scaled trajectory for global alignment to compute the ATE.

The ATEs for different odometry schemes are listed in Table I. The proposed DBA-Fusion shows good performance on most of the sequences, which is more significant on the “outdoor” sequences. This could be attributed to the superiority of the deep optical flow to track environmental features even in low-texture environments, as well as the good compatibility of its iterative re-weighting mechanism in a VIO system. Besides, the trajectories of typical odometry schemes on “outdoors1” and “outdoors7” are shown in Fig. 7. The proposed method shows dramatically better translation and attitude estimation

TABLE II
RELATIVE POSE ERRORS OF DIFFERENT VIO SCHEMES ON KITTI-360 DATASET. BEST MONOCULAR RESULTS ARE UNDERLINED

Seq.	Desc.	VINS stereo		VINS mono		ORB mono		DM-VIO mono		DROID [*] mono		DBA mono	
		t_{rel}^1	r_{rel}^1	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}	t_{rel}	r_{rel}
0000	Suburb	0.626	0.212	1.897	0.176	2.386	0.117	1.369	0.129	17.646	0.205	<u>0.678</u>	0.105
0002	Suburb	0.591	0.169	1.006	0.199	1.309	0.215	0.724	0.183	8.031	0.284	0.577	0.174
0003	Highway	0.502	0.073	2.754	0.088	7.044	0.151	1.146	0.111	22.473	0.144	<u>1.041</u>	0.114
0004	Suburb	0.692	0.172	1.710	<u>0.193</u>	1.976	0.211	1.063	0.178	13.636	0.495	0.556	0.153
0005	Suburb	0.607	0.207	1.187	0.219	1.414	0.227	0.729	0.224	6.969	0.410	<u>0.619</u>	0.209
0006	Suburb	0.630	0.149	1.349	0.176	1.685	0.184	0.887	<u>0.161</u>	10.673	0.209	<u>0.734</u>	0.166
0009 ²	Suburb	0.486	0.121	1.596	0.144	2.407	0.184	1.379	0.136	13.054	0.275	<u>0.846</u>	0.136
0010	Boulevard	0.776	0.206	3.610	0.216	5.335	0.214	2.130	0.215	20.522	0.258	<u>1.486</u>	<u>0.208</u>

^{*} Visual-only. ¹ t_{rel} in %, r_{rel} in $^{\circ}/100$ m. ² Due to the gap of IMU data, only the first 1100 seconds are used. Best results in bold and best monocular results underlined.

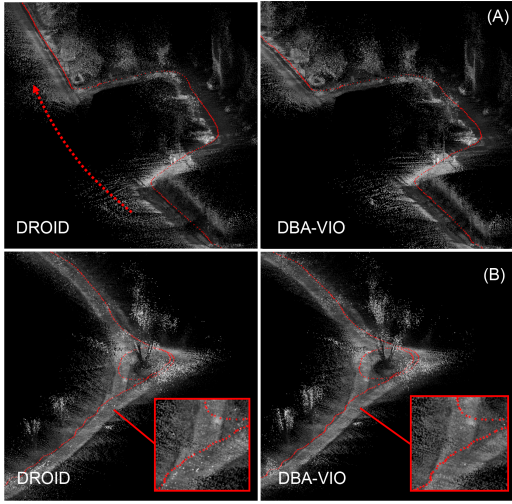


Fig. 8. Dense mapping performance at the regions annotated in Fig. 7. For DROID-SLAM, pointcloud bending and misalignment occur in (A) and (B) respectively, while DBA-VIO achieves good local map consistency.

than the visual-only DROID-SLAM, verifying the contribution of IMU integration to maintain low-drifting, metric-scale pose estimation. Compared to the VINS-Fusion (stereo) scheme, the proposed method obtains comparable scale estimation and shows better attitude maintenance.

As to online dense mapping, two scenes reconstructed by DROID-SLAM and DBA-VIO are shown in Fig. 8. For DROID-SLAM in online mode, the bending and misalignment of the pointclouds indicate significant pose drift, while this is almost overcome by DBA-VIO with the tight integration of a low-cost IMU. It is interesting that the low-drifting pose estimation of DBA-VIO has resulted in local consistent mapping performance without any loop closure techniques.

B. KITTI-360 Dataset

The KITTI-360 benchmark [33] is an autonomous driving dataset comprising multi-sensor data collected at the suburbs of Karlsruhe, Germany on 2013/5/28. The scenarios mainly include narrow suburban roads and highways, with relatively large scale and long durations. Among the mounted sensors, we use the perspective cameras and the OXTS IMU for the test. The georegistered vehicle trajectory obtained by multi-sensor fusion is taken as the reference.

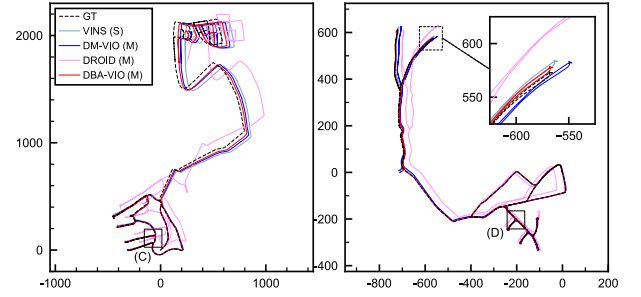


Fig. 9. Trajectories of typical odometry schemes on “0004” and “0005”. The trajectories are aligned with the ground-truth pose at $t = 10$ s. The trajectory of DROID-SLAM (M) is scaled.

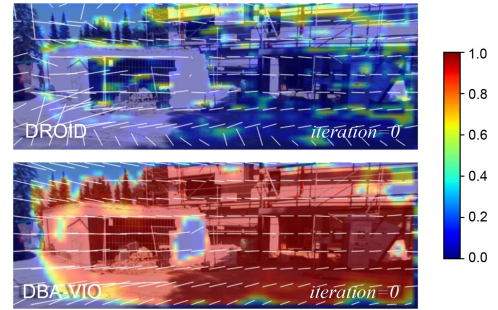


Fig. 10. Optical flow produced by DROID-SLAM and DBA-VIO at the first iteration for one image pair in “0002” sequence of KITTI-360. The heat map indicates the weight (inverse uncertainty) of the optical flow.

Based on the high-precision ground-truth poses, we evaluate the odometry performance of different schemes (as mentioned in Section V-A) through relative pose errors, following [35]. For the visual-only DROID-SLAM, we use the scaled trajectory obtained by whole trajectory $SIM(3)$ alignment. The statistics are listed in Table II. As shown in the table, VINS-Fusion (stereo) obtains the best performance, as the relatively long stereo baseline (≈ 0.6 m) provides strong geometric information. Among the monocular schemes, the proposed DBA-VIO obtains superior performance, which is also indicated by the trajectories in Fig. 9. The improvement of introducing IMU into DROID-SLAM is dramatic, which maximizes the potential of the dense-indirect visual frontend by providing good robustness. As an indicator, Fig. 10 shows the optical flow tracking performance with or without IMU when the vehicle turned left. With IMU aiding, the optical flow module can generate good

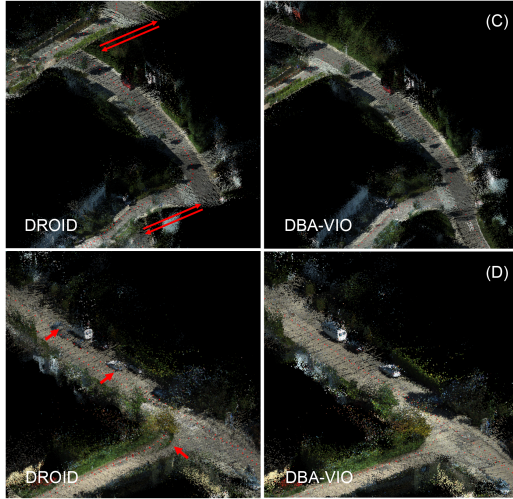


Fig. 11. Dense mapping performance at the regions annotated in Fig. 9. For DROID-SLAM, severe misalignment and ghosting of the pointcloud is seen in (C) and (D) respectively.

results even at the first iteration, thus contributes to better DBA convergence.

We also show the online mapping results in Fig. 11. The results are consistent with Section V-A, as the proposed DBA-VIO achieves locally consistent mapping performance based on the accurate pose estimation and significantly improves the practicability of DROID-SLAM in outdoor scenarios.

C. Self-Made Urban Dataset

To further evaluate the large-scale localization and mapping performance of the system, we turn to the self-made multi-sensor dataset collected in Wuhan City on 2022/10/12. The experimental vehicle is equipped with RGB cameras, an ADIS16470 IMU and a Septentrio AsterRx4 GNSS receiver. A nearby base station is used for differential GNSS (DGNSS) processing. The smoothed trajectory of post-processing DGNSS/INS integration based on high-end IMU is taken as reference. The reference solution is also used to simulate 1 Hz wheel speed measurements with 5 cm/s noise. The data sequence is made publicly available. The trajectories and horizontal position errors of different navigation schemes are shown in Fig. 12. As shown in the figure, the GNSS RTK scheme achieves driftless positioning but undergoes large errors during GNSS occlusion. For DBA-VIO, the position error drifts significantly at the beginning 100 seconds as the IMU excitation is insufficient. With wheel speed integration, the DBA-Fusion (W) scheme obtains smoother pose estimation performance with instant scale awareness. With GNSS RTK integration, the DBA-Fusion (G) scheme achieves stable decimeter-level position estimation. The result outperforms VINS-Fusion (G), which could be attributed to that the GNSS position information is tightly integrated and effectively constrains the VIO subsystem.

Besides, the mapping performance of DBA-Fusion (G) is shown in Fig. 13. Generally, the proposed system could generate dense, accurate and georeferenced point clouds. As shown in Fig. 13(h) and (j), the mapping consistency is good when traversing the same area repeatedly, benefiting from the accurate global pose estimation. The dynamic objects are mostly filtered through multi-frame depth checking, leading to the black vacuum in

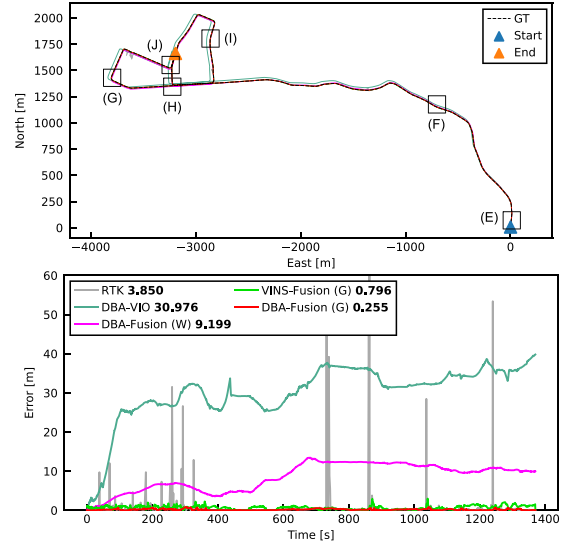


Fig. 12. The trajectories and horizontal position error of different navigation schemes. The bold numbers are root mean square error (RMSE) in meters. The navigation scheme includes: 1) GNSS real-time kinematic (RTK); 2) VINS-Fusion with GNSS; 3) DBA-VIO; 4) DBA-Fusion with WSS; 5) DBA-Fusion with GNSS. All solutions are tested with **monocular** setup.

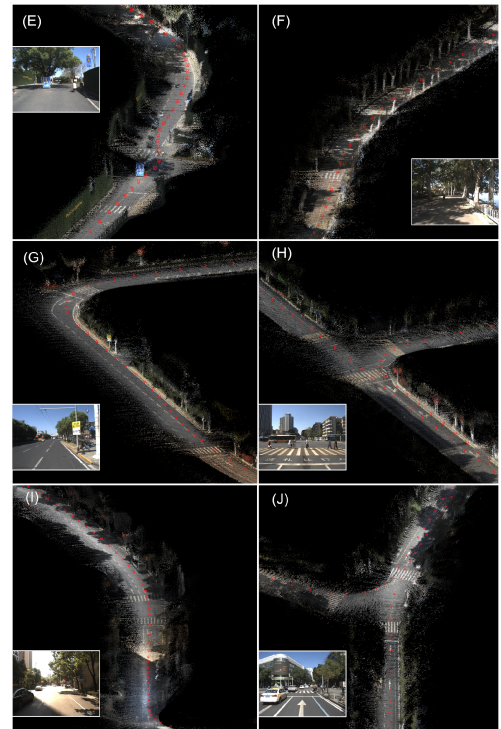


Fig. 13. Mapping performance of DBA-Fusion (G) on the urban dataset.

Fig. 13(j). This is one of the main limitations of the mapping functionality of the system, while this could be partly overcome via multi-camera configuration.

In addition, the real-time performance of the system evaluated on Nvidia RTX 3090 GPU is shown in Table III.

TABLE III
REAL-TIME PERFORMANCE OF THE PROPOSED SYSTEM

Procedure	Time cost (ms)		
	TUM-VI	KITTI-360	Wuhan
Feature Extraction	11.3	10.7	11.6
Graph Management & Correlation	27.2	32.3	25.3
Iteration ¹			
Optical Flow	21.3	21.2	20.0
BA Hessian ³	3.6	4.0	3.6
Optimization ³	3.4	3.4	3.2
Depth Update ³	1.2	1.2	1.2
Marginalization	2.2	1.9	1.4
Real-time Performance ²	1.47x	1.51x	1.62x

¹ Number of iterations is 2 for non-keyframe epoch, 3 for keyframe epoch (see Sect. III-D) and 0 for skipped frames with little disparity.

² Using 5 Hz images as mentioned above.

³ Total time of 2 low-level iterations (see Fig. 3).

V. CONCLUSION

In this letter, we propose a framework that tightly integrates deep dense visual bundle adjustment with multiple sensors in a probabilistic manner, enabling real-time localization and dense mapping. The experimental results validate the superior localization performance of the VIO implementation based on this framework, and show its flexibility to fuse GNSS, WSS for large-scale, georeferenced applications. Future work will focus on extending this system to dynamic scenarios and neural map representations.

ACKNOWLEDGMENT

The implemented DBA-Fusion is developed by the GREAT Group, School of Geodesy and Geomatics, Wuhan University. The numerical calculations in this letter have been done on the supercomputing system at the Supercomputing Center of Wuhan University.

REFERENCES

- [1] C. Cadena et al., "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1309–1332, Dec. 2016.
- [2] K. Wang, S. Ma, J. Chen, F. Ren, and J. Lu, "Approaches, challenges, and applications for deep visual odometry: Toward complicated and emerging areas," *IEEE Trans. Cogn. Develop. Syst.*, vol. 14, no. 1, pp. 35–49, Mar. 2022.
- [3] J. Czarnowski, T. Laidlow, R. Clark, and A. J. Davison, "DeepFactors: Real-time probabilistic dense monocular SLAM," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 721–728, Apr. 2020.
- [4] Z. Teed and J. Deng, "DROID-SLAM: Deep visual SLAM for monocular, stereo, and RGB-D cameras," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 16558–16569.
- [5] T. Qin, P. Li, and S. Shen, "VINS-MONO: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.
- [6] X. Li et al., "Continuous and precise positioning in urban environments by tightly coupled integration of GNSS, INS and vision," *IEEE Robot. Automat. Lett.*, vol. 7, no. 4, pp. 11458–11465, Oct. 2022.
- [7] K. Wu, C. Guo, G. Georgiou, and S. I. Roumeliotis, "Vins on wheels," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 5155–5162.
- [8] D. Li et al., "DXSLAM: A robust and efficient visual SLAM system with deep features," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 4958–4965.
- [9] L. Sun et al., "Improving monocular visual odometry using learned depth," *IEEE Trans. Robot.*, vol. 38, no. 5, pp. 3173–3186, Oct. 2022.
- [10] S. Yang and S. Scherer, "CubeSLAM: Monocular 3-D object SLAM," *IEEE Trans. Robot.*, vol. 35, no. 4, pp. 925–938, Aug. 2019.
- [11] J. Zhang, M. Henein, R. Mahony, and V. Ila, "VDO-SLAM: A visual dynamic object-aware SLAM system," 2020, *arXiv:2005.11052*.
- [12] S. Wang, R. Clark, H. Wen, and N. Trigoni, "DeepVO: Towards end-to-end visual odometry with deep recurrent convolutional neural networks," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 2043–2050.
- [13] J. Jiao, J. Jiao, Y. Mo, W. Liu, and Z. Deng, "MagicVO: An end-to-end hybrid CNN and Bi-LSTM method for monocular visual odometry," *IEEE Access*, vol. 7, pp. 94118–94127, 2019.
- [14] H. Wang, J. Wang, and L. Agapito, "Co-SLAM: Joint coordinate and sparse parametric encodings for neural real-time SLAM," in *Proc. IEEE/CVF Conf. Computer Vis. Pattern Recognit.*, 2023, pp. 13293–13302.
- [15] Z. Zhu et al., "NICER-SLAM: Neural implicit scene encoding for RGB SLAM," in *Proc. Int. Conf. 3D Vis.*, 2024.
- [16] A. I. Mourikis and S. I. Roumeliotis, "A multi-state constraint Kalman filter for vision-aided inertial navigation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2007, pp. 3565–3572.
- [17] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, "Keyframe-based visual-inertial odometry using nonlinear optimization," *Int. J. Robot. Res.*, vol. 34, no. 3, pp. 314–334, 2014.
- [18] M. Li and A. I. Mourikis, "Improving the accuracy of EKF-based visual-inertial odometry," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2012, pp. 828–835.
- [19] L. Stumberg, V. Usenko, and D. Cremers, "Direct sparse visual-inertial odometry using dynamic marginalization," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 2510–2517.
- [20] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.
- [21] L. Stumberg and D. Cremers, "DM-VIO: Delayed marginalization visual-inertial odometry," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 1408–1415, Apr. 2022.
- [22] W. Zhang, S. Wang, X. Dong, R. Guo, and N. Haala, "BAMF-SLAM: Bundle adjusted multi-fisheye visual-inertial slam using recurrent field transforms," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 6232–6238.
- [23] X. Peng et al., "DVI-SLAM: A dual visual inertial SLAM network," 2023, *arXiv:2309.13814*.
- [24] M. Zhang, X. Zuo, Y. Chen, Y. Liu, and M. Li, "Pose estimation for ground robots: On manifold representation, integration, reparameterization, and optimization," *IEEE Trans. Robot.*, vol. 37, no. 4, pp. 1081–1099, Aug. 2021.
- [25] T. Qin, S. Cao, J. Pan, and S. Shen, "A general optimization-based framework for global pose estimation with multiple sensors," 2019, *arXiv:1901.03642*.
- [26] S. Cao, X. Lu, and S. Shen, "GVINS: Tightly coupled GNSS-Visual-Inertial fusion for smooth and consistent state estimation," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2004–2021, Aug. 2022.
- [27] Z. Teed and J. Deng, "RAFT: Recurrent all-pairs field transforms for optical flow," in *Proc. Comput. Vis. ECCV*, 2020, pp. 402–419.
- [28] W. Wang et al., "Tartanair: A dataset to push the limits of visual SLAM," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, pp. 4909–4916.
- [29] F. Dellaert and M. Kaess, *Factor Graphs for Robot Perception*. Foundations, Hanover, MA, USA: Now, 2017.
- [30] F. Dellaert and GTSAM Contributors, "GTSAM (version 4.2a8), Georgia tech borg lab," 2022. [Online]. Available: <https://github.com/borglab/gtsam>
- [31] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-manifold preintegration for real-time visual-inertial odometry," *IEEE Trans. Robot.*, vol. 33, no. 1, pp. 1–21, Feb. 2017.
- [32] D. Schubert et al., "The TUM VI benchmark for evaluating visual-inertial odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 1680–1687.
- [33] Y. Liao, J. Xie, and A. Geiger, "KITTI-360: A novel dataset and benchmarks for urban scene understanding in 2D and 3D," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3292–3310, Mar. 2023.
- [34] M. Grupp, "EVO: Python package for the evaluation of odometry and SLAM," 2017. [Online]. Available: <https://github.com/MichaelGrupp/evo>
- [35] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE Conf. Computer Vis. Pattern Recognit.*, 2012, pp. 3354–3361.